

RecSort: The Recursive Sorting Expression Compiler
Language Design & Implementation Report

Michael Calle

Professor Craig Nelson

Penn State Abington

April 30, 2026

1. Language Whitepaper

1.1 Abstract

RecSort is a domain-specific language (DSL) designed to eliminate the boilerplate of writing recursive, multi-key, null-safe sorting functions in Python. By expressing a complex sorting strategy as a concise `.nst` declaration, a developer can compile that declaration into a fully-functional, self-contained Python function in a single command — with no runtime dependency on the RecSort library.

1.2 System Inputs and Outputs

Sorting a flat list in Python is trivial. Sorting nested data — orders each containing items, employees each carrying a skills list, products with multi-level reviews, requires writing the same fragile lambda tower from scratch every time. A typical two-level sort with null-handling routinely spans 10–20 lines of low-value, error-prone code:

```
def sort_orders(orders):
    for o in orders:
        o["items"] = sorted(
            o.get("items") or [],
            key=lambda x: (x.get("price") is None, x.get("price"))
        )
    return sorted(orders, key=lambda o: (
        o.get("status") is None, o.get("status"),
        -(o.get("date") or 0)
    ))
```

This code is repetitive, difficult to read, and error-prone — especially as nesting depth or number of sort keys increases.

1.3 Proposed Solution

RecSort replaces that boilerplate with a concise, readable declaration:

```
sort orders
  by status asc, date desc
  then items
    by price asc
  nulls last
```

The compiler reads this .nst file and emits ready-to-import Python — type-annotated, commented, null-safe, and fully independent of RecSort at runtime.

1.4 Design Goals

- Expressiveness: capture arbitrarily deep recursive sorts in minimal syntax
- Safety: validate keys, detect cycles and duplicate declarations at compile time
- Portability: generated code depends only on Python stdlib
- Transparency: --preview mode explains the compiled plan in plain English
- Extensibility: multi-block files, dot-notation paths, group-by and limit modifiers

1.5 Target Audience

RecSort targets Python developers who regularly work with nested JSON or dict-based data — particularly in data pipeline, ETL, and API integration contexts.

2. Language Specification Document

2.1 File Format

RecSort source files carry the .nst extension. Files are UTF-8 plain text.

Indentation is significant and must use spaces (not tabs). Comments begin with # and extend to the end of the line. Blank lines are ignored. A single file may contain one or more sort blocks.

2.2 Novel Contributions

```
file          ::= sort_block+
sort_block    ::= 'sort' IDENT NEWLINE clause*
clause        ::= by_clause | then_clause | nulls_clause
                  | stable_clause | limit_clause
                  | group_by_clause | on_missing_clause
by_clause     ::= INDENT 'by' key_dir (',' key_dir)* NEWLINE
key_dir       ::= KEY_PATH ('asc' | 'desc')?
then_clause   ::= INDENT 'then' IDENT NEWLINE clause*
nulls_clause  ::= INDENT 'nulls' ('first' | 'last') NEWLINE
stable_clause ::= INDENT 'stable' NEWLINE
limit_clause  ::= INDENT 'limit' INTEGER NEWLINE
group_by_clause ::= INDENT 'group-by' IDENT NEWLINE
on_missing_clause ::= INDENT 'on-missing' ('skip' | 'stop' | 'log') NEWLINE
KEY_PATH      ::= IDENT ('.' IDENT)*
IDENT         ::= [a-zA-Z_][a-zA-Z0-9_]*
```

2.3 Keyword Reference

Keyword	Description	Default
sort <name>	Begins a sort block; names the generated function sort_<name>	Required
by <key> asc desc	Sort key and direction. Chain multiple with commas.	Required

then <list-key>	Sub-sort: for each item, sort the nested list at item[<list-key>].	Optional
nulls first last	Placement of None values in the sort output.	last
stable	Documents that equal elements preserve original order.	False
limit <N>	Truncate sorted result to N items.	None
group-by <key>	Return a dict keyed by field instead of a flat list.	None
on-missing skip stop log	Behaviour when a record is missing a sort key.	treat as null

2.4 Dot-Notation Key Paths

Any key in a by clause may use dot notation to access nested dict fields: by customer.name asc. The compiler emits a null-safe chained accessor: (`_x.get('customer') or {}`).get('name'). This prevents `KeyError` on missing intermediate levels.

2.5 Multi-Block Files

A single .nst file may contain any number of sort blocks, each starting with a sort keyword at column 0. Each block compiles to its own function in the output .py file. Block names within a single file must be unique (enforced by semantic analysis).

3. Language Tutorial

3.1 Module Overview

Install RecSort from source:

```
git clone https://github.com/MikeJCalle/RecSort
cd recsort && pip install -e .
```

3.2 Hello, RecSort

Create a file named simple.nst:

```
sort products
  by price asc
```

Compile it:

```
sort products
  by price asc
```

Use it in Python:

```
recsort simple.nst
# Output: simple.py (functions: sort_products)
```

Use it in Python:

```
from simple import sort_products
data = [{"price": 9.99}, {"price": 2.50}, {"price": 14.00}]
result = sort_products(data) # [{price:2.50}, {price:9.99}, {price:14.00}]
```

3.3 Multi-Key Sorts

Sort employees by department ascending, then hire date ascending within each department:

```
sort employees
  by department asc, hire_date asc
  nulls last
```

3.4 Nested (Recursive) Sorts

Sort employees by department ascending, then hire date ascending within each department:

```
sort orders
  by status asc, date desc
  then items
    by price asc
  nulls last
```

then blocks nest to any depth. Each generates its own inner sort loop in the output function.

3.5 Dot-Notation Keys

Navigate nested dicts safely without manual traversal:

```
sort customers
  by address.city asc, meta.score desc
```

3.6 Nested (Recursive) Sorts

group-by:

```
sort products
  by rating desc
  group-by category
```

Returns a dict mapping category name to its sorted list of products.

limit:

```
sort products
  by rating desc
  limit 10
```

Returns only the top 10 rated products.

on-missing:

```
sort orders
  by customer_id asc
  on-missing skip
```

Records missing `customer_id` are filtered out. Other options: `stop` (raises `KeyError`) and `log` (warns + keeps record).

3.7 Preview Mode

Before writing any file, verify the plan in plain English:

```
recsort orders.nst --preview
```

3.8 Preview Mode

Before writing any file, verify the plan in plain English:

```
recsort orders.nst --check sample_data.json
```

The checker reports null counts per key and verifies the output is correctly ordered.

4. Language Reference Manual

4.1 CLI Reference

Command	Description
recsort <file.nst>	Compile file.nst → file.py
recsort <file.nst> --preview	Print plain-English plan; do not write file
recsort <file.nst> --output <path>	Write compiled output to a specific path
recsort <file.nst> --function-name <n>	Override the generated function name
recsort <file.nst> --check <data.json>	Compile and validate against real JSON data
recsort --demo	Run built-in demo with sample data
recsort --help	Show usage information

4.2 Exit Codes

Code	Meaning
0	Success
1	Parse error
2	Semantic error
3	IO error (file not found, write failure)
4	Check failed (sort validation found issues)

4.3 Generated Python API

For flat sorts, the generated function signature is:

```
def sort_<name>(data: list[dict]) -> list[dict]
```

For group-by sorts:

```
def sort_<name>(data: list[dict]) -> dict[str, list[dict]]
```

Generated functions are self-contained — they include an internal `_Desc` helper class and require no imports beyond Python stdlib.

4.4 Semantic Rules

1. Every sort block must contain at least one by clause.
2. Duplicate key names within a single by clause are a semantic error.
3. Duplicate sort block names within a file are a semantic error.
4. then block keys must resolve to list fields in the parent scope.
5. group-by on a key that also appears in the by clause raises a warning.

6. A sort block with no by clause raises a SemanticWarning (not an error).

4.5 Null Handling Semantics

When nulls last (default), records where the sort key is None sort after all non-null records. When nulls first, they sort before. The null sentinel used in comparisons is type-aware: numeric keys default to 0 and string keys default to "".

5. Project Development Plan

5.1 Toy Network Definition

RecSort was developed by Michael Calle as a single-author compiler project. The goal was to produce a complete, working DSL compiler, lexer, parser, semantic analysis, code generator, CLI, and test suite in Python, compiled and distributed as a standard package.

5.2 Hand Calculation — Dijkstra (Vehicle Mobility)

Phase	Milestone	Deliverable
v0.1 Core	Lexer, parser, flat sort codegen, basic CLI	Functional MVP compiler
v0.1 Tests	Parser unit tests, semantic unit tests	test_parser.py, test_semantic.py
v0.1 Integration	End-to-end compile + execute tests	test_integration.py

v0.2 Depth	Arbitrary nesting, dot notation, type sentinels	compiler.py v0.2
v0.2 Features	limit, group-by, on-missing, stable	Updated codegen
v0.2 CLI	--output, --function-name, --check, exit codes	Updated main()
v0.2 Tests	deep_nesting, dot_notation, final_features suites	Three new test files
v0.2 Docs	README, CONTRIBUTING, examples/	Documentation files

5.3 Tools and Technologies

- Language: Python 3.11+
- Development Environment: PyCharm with step-through debugger
- Version Control: Git (GitHub repository: MikeJCalle/RecSort)
- Testing Framework: pytest
- CI/CD: GitHub Actions (.github/workflows/ci.yml)
- Package Management: pyproject.toml (PEP 517 build)
- Source files: .nst (plain text, diffable)

5.4 BFS Reachability Verification

```
recsort/
  recsort/
    __init__.py      # Public API exports
    __main__.py      # python -m recsort entry point
    compiler.py      # All compiler stages
  tests/
    test_parser.py
    test_semantic.py
    test_integration.py
    test_deep_nesting.py
```

```
test_dot_notation_and_sentinel.py
test_final_features.py
examples/
  orders.nst
  products.nst
  employees.nst
.github/workflows/ci.yml
pyproject.toml
README.md
CONTRIBUTING.md
```

6. Language Evolution

Four test scenarios were executed against the full sample scenario graph (8 locations, 11 bidirectional road pairs, 3 safe zones, 3 civilian zones). The sample scenario represents a conflict-zone geography with villages, a town centre, a checkpoints, a field medical station, and three safe zones of varying capacity.

6.1 Version 0.1.0 — Initial Release

The initial version established the core of RecSort: a working lexer, parser, semantic analyser, and code generator for single-block .nst files. v0.1 supported:

- Single sort block per file
- Flat multi-key sorts (by key1 asc, key2 desc)
- One level of then nesting
- nulls first/last modifier
- Basic CLI: compile to .py

6.2 Version 0.2.0 — Feature Expansion

New Language Features

- Arbitrary-depth nesting — then blocks now nest to any depth. The code generator uses unique loop variables (`_i0`, `_i1`, ...) to prevent shadowing.
- Dot-notation key access — by `customer.name` asc compiles to safe chained `.get()` calls.
- `stable` keyword — makes the stability guarantee explicit and visible in preview output.
- `limit <N>` — truncate sorted results at any nesting level.
- `group-by <key>` — return a dict grouped by a field instead of a flat list.
- `on-missing skip|stop|log` — control behaviour for records missing a sort key.
- Multiple sort blocks per file — each compiles to its own function.

New CLI Flags

- `--output <path>`: write compiled `.py` to any location
- `--function-name <name>`: override the generated function name
- `--check <data.json>`: compile and validate against real data
- Structured exit codes (0–4) for CI integration
-
- Semantic Analysis Improvements

- Duplicate sort key detection within a by clause
- Missing root by clause warning
- Duplicate block name detection across a file
- group-by redundancy warning

Code Generation Improvements

- Type annotations on generated functions
- Type-aware null sentinels (0 for numeric keys, " for strings)
- Unique loop variables for deep nesting

6.3 Bug Fixes in v0.2

- Dot-notation keys previously compiled to literal string lookups (always returned None). Fixed to emit chained `.get()` accessors.
- All-null lists on numeric keys raised `TypeError` during sort comparison. Fixed by type-aware sentinels.
- then blocks deeper than one level were silently dropped. Fixed by recursive emission with `_emit_children_only`.

6.4 Scenario 3: BFS Reachability from Beit Nour Village

- PyPI publication for `pip install recsort`
- Support for list-of-lists (sort keys that are themselves lists)

- Optional sort key type annotations in .nst syntax
- IDE plugin (VS Code syntax highlighting for .nst files)
- Web playground for live compilation

7. Translator Architecture

7.1 Overview

RecSort is a source-to-source compiler (transpiler). It reads a .nst source file and emits Python source code. The translation pipeline consists of four distinct stages:

Stage	Module	Input	Output
Lexer / Tokeniser	<code>_tokenise()</code> , <code>_indent()</code>	.nst source text	Token list per line
Parser	<code>parse()</code>	Token lines	List of SortBlock AST nodes
Semantic Analysis	<code>analyse()</code> , <code>_check_block()</code>	SortBlock list	Validated SortBlock list or error
Code Generator	<code>generate()</code> , <code>_emit_block()</code>	Validated SortBlock	Python source string

7.2 Lexer

The lexer is line-based and indent-sensitive. Each non-blank, non-comment line is processed by two functions: `_indent()` measures leading-space depth, and `_tokenise()` splits the stripped content into tokens while stripping trailing commas (to support comma-separated by clauses naturally). The lexer does not produce a flat token stream;

instead it produces a list of (lineno, indent, tokens) triples consumed directly by the parser.

7.3 Parser

- The parser is an iterative, stack-based recursive descent parser. A stack of (indent_level, SortBlock) pairs tracks the current nesting context. When a new line is read, the stack is popped until the top entry has a shallower indent than the current line, identifying the parent block. The parser supports these verbs: sort (block header), by (key declarations), then (child block), nulls, stable, limit, group-by, on-missing.
- Multi-block files are handled by detecting a top-level sort token while still inside an existing block — the current block is closed and a new root is started.
- The parser produces a list of SortBlock dataclass instances, each containing a list of SortKey objects and a children list for nested then blocks.

7.4 Semantic Analysis

analyse() walks the list of root SortBlock nodes and calls _check_block() recursively. Checks performed:

- Duplicate sort block names across the file (SemanticError)
- Missing by clause on a sort block (SemanticWarning)
- Duplicate key within a single by clause (SemanticError)
- group-by key also present in the by clause (SemanticWarning)
- Recursive descent into children for all of the above

7.5 Code Generator

The code generator traverses the validated SortBlock tree and emits Python source text. Key functions:

- `generate(root)` — top-level entry; emits the runtime header + function body
- `_emit_block(block, data_var, pad, depth)` — emits the `sorted()` call and children
- `_emit_children_only(block, ...)` — recursively emits inner for loops for then blocks
- `_emit_on_missing_guard(block, ...)` — emits skip/stop/log filter before the sort
- `_emit_sorted_call(block, ...)` — constructs the `key= lambda` for the `sorted()` call
- `_key_expr(sk, nulls, item_var)` — builds the null-aware comparison tuple for one key
- `_dot_access(item_var, key)` — emits chained `.get()` for dot-notation paths
- `_type_sentinel(key)` — infers the null fallback value from the key name

7.6 Runtime Helper

The generated Python includes a small `_Desc` class that inverts comparison operators to achieve descending sort without negation — making it work correctly for strings, dates, and other non-numeric types. This class is included in every output file and has no external dependencies.

8. Component Developers

8.1 Project Authorship

RecSort was designed, implemented, tested, and documented by a single developer:

Role	Developer	Responsibilities
Language Designer	Michael Calle	DSL syntax, keyword design, semantic rules
Compiler Author	Michael Calle	Lexer, parser, semantic analysis, code generator
CLI Engineer	Michael Calle	CLI flags, exit codes, --check and --preview modes
Test Engineer	Michael Calle	All test suites (parser, semantic, integration, deep nesting, dot notation, final features)
Documentation Author	Michael Calle	README, CONTRIBUTING, examples, this report

8.2 Version Timeline

Version 0.1.0 was written on April 5, 2026. Version 0.2.0 was completed on April 30, 2026. Both versions were developed and committed to the GitHub repository at <https://github.com/MikeJCalle/RecSort>.

8.3 Development Toolchain

The compiler was developed entirely in Python using PyCharm as the IDE. PyCharm's integrated debugger was used extensively to step through lexer and parser

state during development. The CI pipeline runs on GitHub Actions, executing the full pytest suite on every push.

9. Development Environment and Runtime

9.1 Development Environment

Component	Details
Language	Python 3.11+
IDE	PyCharm (step-through debugging)
Version Control	Git / GitHub
CI System	GitHub Actions
Package Format	PEP 517 (pyproject.toml)
Test Runner	pytest
Source File Type	Plain-text .nst files (UTF-8, diffable)

9.2 Runtime Requirements

RecSort's generated Python output has zero runtime dependencies beyond the Python standard library. The generated .py file is completely self-contained and can be dropped into any Python 3.8+ project.

The RecSort compiler itself requires only Python 3.8+ with no third-party packages. All standard library modules used (json, sys, textwrap, warnings, dataclasses, pathlib) are available in any supported Python version.

9.3 Installation

```
# From source (development install):  
git clone https://github.com/MikeJCalle/RecSort  
cd recsort  
pip install -e .  
  
# Once published to PyPI:  
pip install recsort
```

9.4 CI/CD Pipeline

The GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push and pull request. It installs the package in development mode and executes `pytest` across the full test suite. Named exit codes enable the CI system to distinguish parse errors, semantic errors, and IO failures.

9.5 Package Configuration

The `pyproject.toml` file defines the package metadata, entry points, and build configuration. The `recsort.compiler:main` function is registered as the `recsort` command-line entry point. The package exposes a public Python API through `recsort/__init__.py`, exporting `parse`, `analyse`, `generate`, `compile_nst`, `preview`, `ParseError`, and `SemanticError`.

10. Test Plan, Test Scripts, and Suites

10.1 Test Strategy

RecSort follows a layered testing strategy. Each compiler stage has its own unit tests. Integration tests verify the full compile → execute → assert pipeline. Feature tests verify each v0.2 addition in isolation.

10.2 Test Suites

File	Stage	Coverage
test_parser.py	Lexer + Parser	Token types, indent detection, valid/malformed .nst lines, SortNode tree structure, flat/nested/multi-block
test_semantic.py	Semantic Analysis	Dangling sub-sort detection, key-path cycle detection, direction inconsistency, duplicate names and warnings
test_integration.py	End-to-End	Compile → import → execute flow; ordering assertions at every nesting level; empty lists, all-null cols, length-1
test_deep_nesting.py	v0.2 Depth	3+ levels of then nesting; loop variable uniqueness; deep null handling
test_dot_notation_and_sentinel.py	v0.2 Dot+Sentinel	Dot-notation key compilation; chained .get() emission; numeric vs string sentinel selection; all-null numeric lists
test_final_features.py	v0.2 Features	limit, group-by, on-missing (skip/stop/log), stable, multi-block files, --check validation, structured exit codes

10.3 Sample Test Cases

Parser: Flat Single Key

```
def test_parse_flat_single_key():  
    src = "sort orders\n  by date asc\n"
```

```
root = parse(src)
assert root.name == "orders"
assert root.keys[0] == SortKey(key="date", direction="asc")
assert root.nulls == "last"    # default
```

Integration: Nulls Last Default

```
def test_nulls_last_default():
    src = "sort items\n  by value asc\n"
    data = [{"value": 2}, {"value": None}, {"value": 1}]
    result = _run(src, data)
    assert result[-1]["value"] is None
```

Integration: Nested Two-Level

```
def test_nested_two_level():
    src = ("sort orders\n  by status asc\n"
          "  then items\n    by price asc\n")
    data = [
        {"status": "b", "items": [{"price": 3}, {"price": 1}]},
        {"status": "a", "items": [{"price": 9}, {"price": 2}]},
    ]
    result = _run(src, data)
    assert result[0]["status"] == "a"
    assert [i["price"] for i in result[0]["items"]] == [2, 9]
```

10.4 Running the Tests

```
pip install pytest
pytest                                # run all tests
pytest tests/test_parser.py          # single suite
pytest -v                             # verbose output
```

11. Conclusions

11.1 Summary of Achievements

RecSort v0.2 delivers a complete, working DSL compiler for recursive Python sort functions. Starting from the observation that multi-level sorted() calls are repetitive and error-prone, the project produced:

- A clean, expressive DSL syntax (.nst) that captures arbitrarily nested sort strategies in a few lines
- A full compiler pipeline: lexer → parser → semantic analysis → code generator
- Generated Python that is self-contained, type-annotated, null-safe, and requires no runtime dependency
- A feature-rich CLI with --preview, --check, --output, --function-name, and structured exit codes
- A comprehensive test suite of five test files covering all compiler stages and all v0.2 features
- A CI/CD pipeline on GitHub Actions

11.2 Key Technical Decisions

Several design decisions proved important to the final quality of the project. The choice to generate self-contained Python with no runtime dependency was central: it

means users can compile once and distribute the .py file anywhere without requiring RecSort to be installed in the target environment.

The type-aware null sentinel system (inferring 0 for numeric keys and "" for strings) addressed a subtle but real bug where all-null lists on integer fields would raise `TypeError` during comparison.

Using an iterative, stack-based parser rather than a recursive descent parser simplified the handling of multi-block files and arbitrary nesting depth without risking Python's recursion limit.

11.3 Lessons Learned

Building a complete compiler — even a small DSL — requires careful attention to error propagation. The distinction between parse errors (structural) and semantic errors (logical) proved valuable: different exit codes let CI pipelines give more actionable feedback.

Preview mode (`--preview`) turned out to be one of the most useful features during development, as it provided a fast sanity check before committing to file output.

11.4 Future Directions

The most impactful next steps would be PyPI publication for easy installation, followed by an IDE extension for .nst syntax highlighting. Longer-term, support for schema-aware compilation — where RecSort can read a JSON schema and validate key names — would substantially reduce user errors.

12. Code Listing

12.1 recsort/compiler.py (Selected Sections)

Data Types

```
@dataclass
class SortKey:
    key: str
    direction: str          # 'asc' or 'desc'
    key_type: str = 'auto'

@dataclass
class SortBlock:
    name: str
    keys: list               # list[SortKey]
    nulls: str               # 'first' | 'last'
    stable: bool = False
    limit: int = None
    group_by: str = None
    on_missing: str = 'null'
    children: list = field(default_factory=list)
    parent_key: str = None
```

Lexer

```
def _tokenise(line):
    return [t.rstrip(',') for t in line.split() if t.rstrip(',') ]

def _indent(line):
    return len(line) - len(line.lstrip(' '))
```

Dot-Notation Accessor

```
def _dot_access(item_var, key):
    parts = key.split('.')
    if len(parts) == 1:
        return f"{item_var}.get('{key}')
```

```

expr = item_var
for part in parts[::-1]:
    expr = f"({expr}.get('{part}')) or {{{}}}"
return f"{expr}.get('{parts[-1]}')"
```

Type-Aware Null Sentinel

```

_NUMERIC_KEYS = re.compile(
    r'(?i) (price|amount|total|count|score|rating|rank|
           weight|age|year|month|day|size|qty|quantity|
           _id|_num|_no|_count|_date|latitude|longitude) '
)

def _type_sentinel(key):
    leaf = key.split('.')[ -1]
    return '0' if _NUMERIC_KEYS.search(leaf) else ''
```

Key Expression Builder

```

def _key_expr(sk, nulls, item_var='_x'):
    access = _dot_access(item_var, sk.key)
    sentinel = _type_sentinel(sk.key)
    null_first = (nulls == 'first')
    is_null = f"({access} is None)"
    not_null = f"not ({access} is None)"
    null_flag = f"(False if {is_null} else True)" # first
    if not null_first:
        null_flag = f"(True if {is_null} else False)" # last
    val = f"({access} if {not_null} else {sentinel})"
    if sk.direction == 'desc':
        val = f"_Desc({access} if {not_null} else {sentinel})"
    return f"({null_flag}, {val})"
```

Exit Codes

```

EXIT_OK           = 0
EXIT_PARSE_ERR    = 1
EXIT_SEM_ERR      = 2
EXIT_IO_ERR       = 3
EXIT_CHECK_FAIL   = 4
```

12.2 examples/orders.nst

```
# orders.nst - sort orders by status and date, items by price
sort orders
  by status asc, date desc
  then items
    by price asc
  nulls last
```

12.3 examples/employees.nst

```
# employees.nst - sort employees by department and hire date
sort employees
  by department asc, hire_date asc
  then skills
    by level desc
  nulls last
```

12.4 recsort/_init_.py

```
"""RecSort - Recursive Sort Expression Compiler."""

from .compiler import (
    parse, analyse, generate, compile_nst, preview,
    ParseError, SemanticError,
)

__version__ = '0.1.0'
```

12.5 tests/test_integration.py (Excerpt)

```
def _run(nst_source, data, func_name='sort_test'):
    code = compile_nst(nst_source, func_name)
    ns = {}
    exec(code, ns)
    return ns[func_name](copy.deepcopy(data))

def test_flat_asc():
    src = 'sort items\n  by value asc\n'
    data = [{'value': 3}, {'value': 1}, {'value': 2}]
```

```
result = _run(src, data)
assert [r['value'] for r in result] == [1, 2, 3]

def test_nulls_last_default():
    src = 'sort items\n  by value asc\n'
    data = [{'value': 2}, {'value': None}, {'value': 1}]
    result = _run(src, data)
    assert result[-1]['value'] is None
```

For the complete source code, refer to the GitHub repository:

<https://github.com/MikeJCalle/RecSort>